

Task 2: The discount code

An online shop gave a customer the discount code *PROM050*. The code may be used **at most 3 times**: the shop checks that in the code, with a counter. A week later the accountants saw that the code had been used several hundred times, although the limit was 3. How is this possible?

The situation

Every click on *Use code* starts a new thread that calls `redeem()`. The function checks the counter used, verifies the order (slow: stock, price, address) and counts the use.



Ready code (in `coupon.h`, we do not look inside)

Functions

`verify_order()` simulates the shop checking the order (stock, price, address): 100 ms

Constants

`MAX_USES` 3: the code may be used at most 3 times

This bug is not only a wrong number in a report. Here it costs the shop money, and the same mistake in a check of a limit (a number of tries, a budget, a quota) is a **security hole**. Programmers call it *time of check to time of use* (TOCTOU): the program checks a rule at one moment and uses the result a moment later, when the rule may be already broken.

The code

The template is in [Task2](#). This is coupon.cpp:

```

const int CLICKS = 100;           // how many people click "Use code" at the same time

int used = 0;                     // how many times the code was used

// returns true if the discount was given
bool redeem() {
    if (used >= MAX_USES) { // 1. check: is the limit used up?
        return false;
    }
    verify_order();           // (slow)
    used++;                   // 2. act: count this use
    return true;
}

int main() {
    std::vector<std::thread> threads;
    for (int i = 0; i < CLICKS; i++)
        threads.emplace_back(redeem);
    for (auto &t : threads)
        t.join();
    std::cout << CLICKS << " clicks, the code was used " << used
                << " times (limit " << MAX_USES << ")\n";
}

```

In the task folder, make run compiles the program and runs it.

Your tasks

- 1 Does the limit of 3 uses really work? Suppose the code has already been used 2 times, so one use is left. Two customers click *Use code* at the same moment. Describe, step by step, how both of them can get the discount.
- 2 Set CLICKS to 2, 10, 100 and 1000. Run each one 3 times and write the results in a table. How many times was the code used? What do you notice?
- 3 Fix redeem() so that the code works at most 3 times, for any number of clicks.
- 4 verify_order() is slow, and while one click verifies its order under the mutex, all the others wait. It is possible to move verify_order() **outside** the mutex and still allow at most 3 uses. Think how to do it (Task 1, question 4 is similar) and write it.